

软件安全综合实践

实验一：函数调用过程与栈帧

软件安全综合实践

实验一：函数调用过程与栈帧

学院名称：智能与计算学部

专 业：网络空间安全

学生姓名：安孟喆

学 号：3024244059

年 班 级：2024 级 1 班

2026 年 6 月 25 日

1. 实验目的

- 复习操作系统关于函数调用的知识点
- 复习汇编语言中有关寄存器的知识点
- 学习使用编译器调试功能
- 为软件安全课程准备实验环境

2. 过程记录

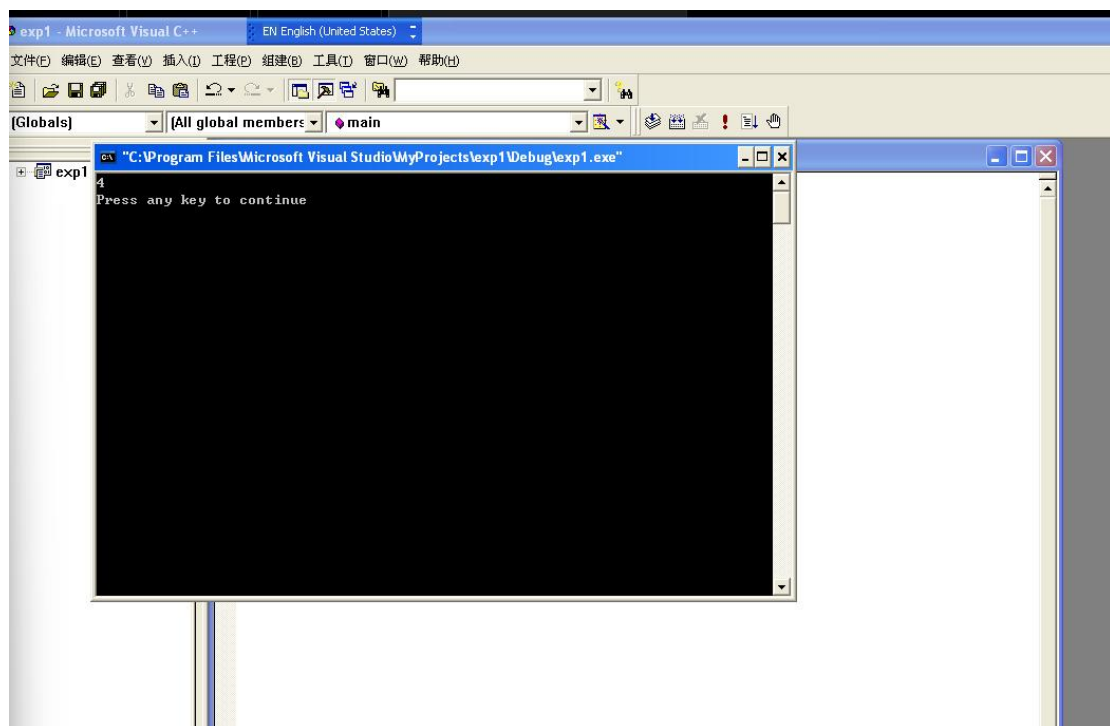


图 1 程序运行结果

2.1 实验环境准备

本次实验在 Windows XP 虚拟机环境下进行，使用 Microsoft Visual C++ 6.0 作为开发和调试工具。实验通过编写简单的 C 语言函数调用程序，在调试模式下观察函数调用过程中栈帧的变化情况。

2.2 实验代码

本次实验使用的 C 语言代码如下，实现了一个简单的整数加法函数调用：

```
C
实验代码 test1.cpp#include "stdafx.h"

int add(int x, int y){
    int z = 0;
    z = x+y;
    return z;
}

int main(int argc, char* argv[]){
    int n = 0;
    n = add(1,3);
    printf("%d\n", n);
    return 0;
}
```

2.3 实验步骤与观察

步骤 1-4：环境搭建与编译

1. 安装 XP 虚拟机
2. 安装 VC 6.0 编译器
3. 编写 C 文件
4. 调试模式进行编译

程序运行结果输出为 4，验证了 add(1,3)函数正确返回了两数之和。

步骤 5-7：设置断点与进入反汇编

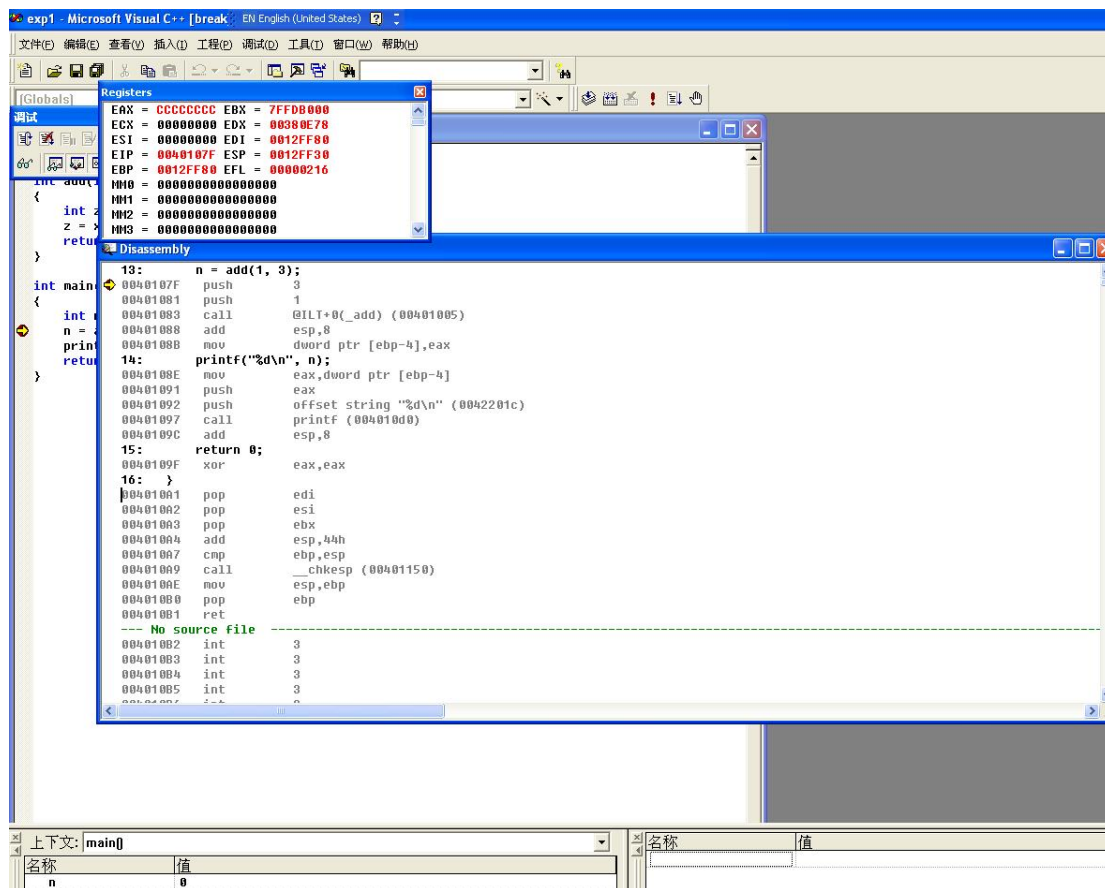


图 2 调试界面与反汇编窗口

1. 在 main 函数内 "n = add(1,3);" 处设置一个断点
2. 按 F5 进入调试状态
3. 点右键，选择 "转到反汇编"

进入反汇编视图后，可以看到 C 语言代码对应的汇编指令。main 函数中调用 add 函数的汇编代码如下：

asm

```
main 函数中调用 add 的汇编代码 0040107F    push        3
00401081    push        1
00401083    call       @ILT+0(add) (00401005)
00401088    add        esp,8
0040108B    mov       dword ptr [ebp-4],eax
```

步骤 8-10：单步执行与参数入栈观察

1. 点 F11 单步执行
2. 观察寄存器窗口和内存窗口

3. 观察参数入栈

参数入栈过程：

函数调用时，参数按照从右向左的顺序入栈。首先执行 `push 3` 将第二个参数 `y=3` 压入栈中，然后执行 `push 1` 将第一个参数 `x=1` 压入栈中。

观察寄存器和内存变化：

- 执行 `push 3` 后，ESP 寄存器值从 0012FF2C 变为 0012FF28，栈顶地址 0012FF28 处存储的值为 03 00 00 00（小端序表示整数 3）
- 执行 `push 1` 后，ESP 寄存器值变为 0012FF24，栈顶地址 0012FF24 处存储的值为 01 00 00 00（小端序表示整数 1）

步骤 11：返回地址入栈观察

1. 观察返回地址入栈

执行 `call` 指令时，CPU 会自动将下一条指令的地址（返回地址）压入栈中，然后跳转到被调用函数的入口地址。

观察结果：

- 执行 `call` 指令后，ESP 变为 0012FF20
- 栈中 0012FF20 地址处存储的是返回地址 00401088（即 `call` 指令下一条指令 `add esp,8` 的地址）
- EIP 寄存器变为 00401020，即 `add` 函数的入口地址

步骤 12：调整栈帧观察

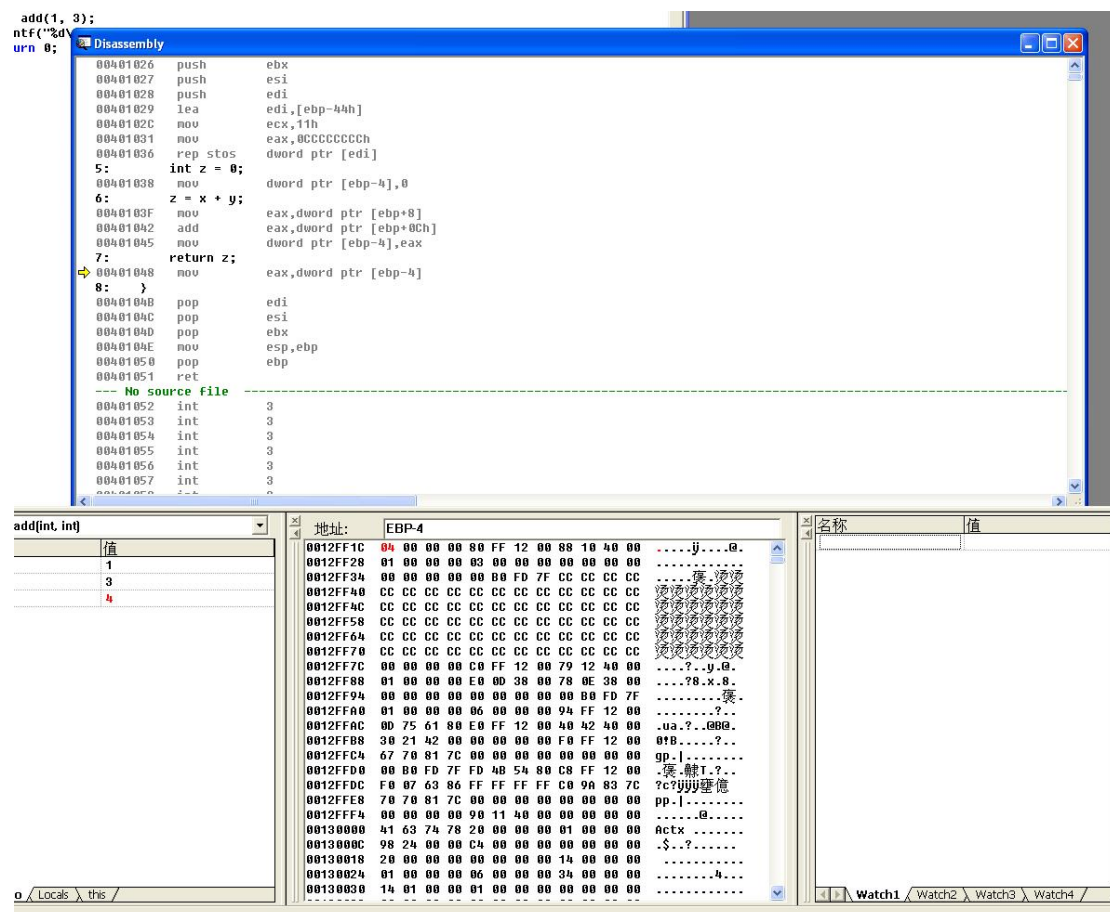


图 3 add 函数栈帧与内存窗口

1. 观察调整栈帧

进入 add 函数后，首先执行栈帧调整的汇编代码：

```
asm
add 函数栈帧调整代码 00401020    push        ebp
00401021    mov         ebp,esp
00401023    sub         esp,44h
00401026    push        ebx
00401027    push        esi
00401028    push        edi
00401029    lea         edi,[ebp-44h]
0040102C    mov         ecx,11h
00401031    mov         eax,0CCCCCCCCh
00401036    rep stos    dword ptr [edi]
```

栈帧调整过程详解：

1. **push ebp**: 保存旧的 EBP 值（main 函数的栈帧基址），ESP 减 4 变为 0012FF1C

- 2. `mov ebp, esp`: 将当前 ESP 值赋给 EBP，建立新的栈帧基址，此时 EBP = 0012FF1C
- 3. `sub esp, 44h`: 为局部变量和临时空间分配栈空间，ESP 减少 68 字节
- 4. `push ebx/esi/edi`: 保存需要保护的寄存器值
- 5. `rep stos dword ptr [edi]`: 将局部变量区域初始化为 0xCFFFFFFF (VC 调试模式的特征，用于检测未初始化变量)

此时栈帧结构从高地址到低地址依次为：

地址（相对 EBP）	内容	说明
EBP + 0Ch	参数 y = 3	第二个参数
EBP + 08h	参数 x = 1	第一个参数
EBP + 04h	返回地址	call 指令的下一条指令地址
EBP + 00h	旧 EBP 值	main 函数的栈帧基址
EBP - 04h	局部变量 z	add 函数的局部变量
EBP - 44h	...	其他临时空间

步骤 13：函数执行观察

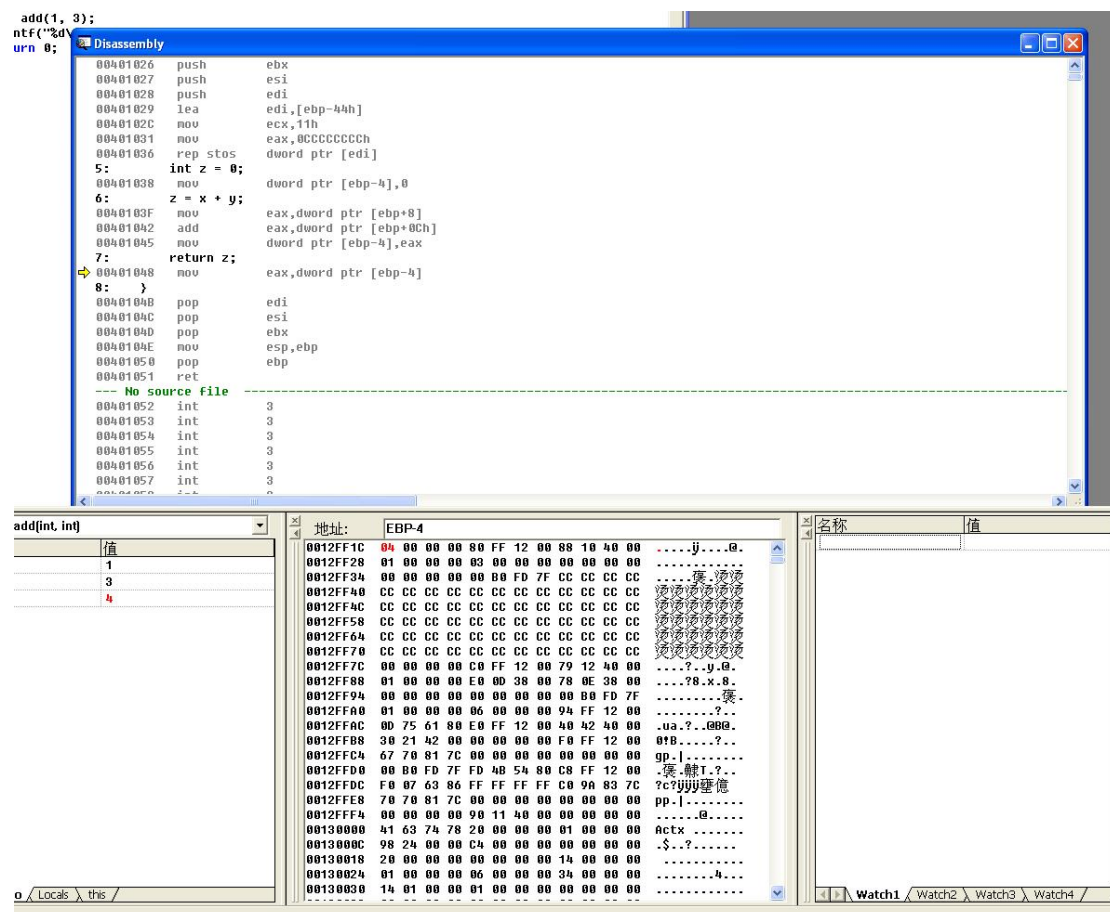


图 4 函数执行过程观察

1. 观察函数执行

函数体执行的汇编代码：

```
asm
add 函数执行代码 00401038    mov          dword ptr [ebp-4],0    ; int
z = 0;
0040103F    mov          eax,dword ptr [ebp+8] ; eax = x
00401042    add          eax,dword ptr [ebp+0Ch]; eax += y
00401045    mov          dword ptr [ebp-4],eax ; z = x + y
00401048    mov          eax,dword ptr [ebp-4] ; 返回值存入 eax
```

执行过程观察：

- 局部变量 `z` 初始化为 0，存储在 `EBP-4` 地址处
- 通过 `[ebp+8]` 访问第一个参数 `x=1`，通过 `[ebp+0Ch]` 访问第二个参数 `y=3`
- 加法运算结果存储在局部变量 `z` 中，值为 4
- 函数返回值通过 `EAX` 寄存器传递，执行完后 `EAX = 00000004`

步骤 14：函数返回观察

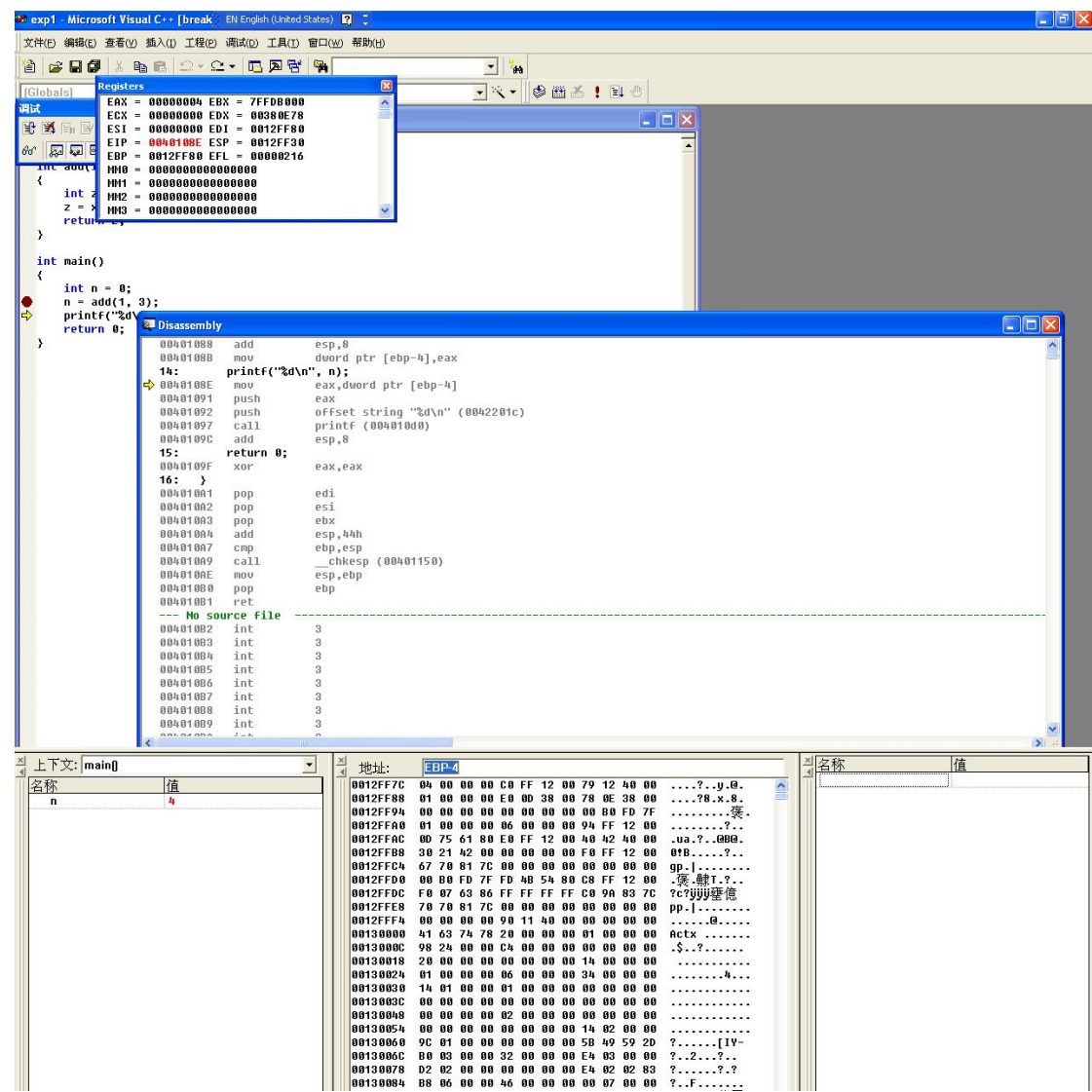


图 5 函数返回后 main 函数状态

1. 观察函数返回

函数返回的汇编代码：

```
asm
add 函数返回代码 0040104B    pop    edi
0040104C    pop    esi
0040104D    pop    ebx
0040104E    mov    esp,ebp
00401050    pop    ebp
00401051    ret
```

函数返回过程详解：

1. `pop edi/esi/ebx`: 恢复之前保存的寄存器值
2. `mov esp, ebp`: 将 ESP 恢复为 EBP 值, 释放局部变量空间
3. `pop ebp`: 恢复旧的 EBP 值 (main 函数的栈帧基址), ESP 增加 4
4. `ret`: 弹出栈顶的返回地址到 EIP, 程序跳回 main 函数继续执行

返回 main 函数后, 执行 `add esp, 8` 平衡栈 (因为两个 int 参数共 8 字节), 然后将 EAX 中的返回值 4 存入局部变量 n 中。

3. 心得体会

通过本次实验, 我深入理解了函数调用过程中栈帧的变化机制, 主要收获如下:

3.1 对函数调用机制的理解

函数调用不是简单的跳转, 而是一个精心设计的栈操作过程。每次函数调用都会在栈上创建一个新的栈帧, 用于保存函数的参数、返回地址、旧栈帧基址和局部变量。这种栈帧机制保证了函数调用的可重入性和递归调用的正确性。

3.2 对寄存器作用的认识

在函数调用过程中, 各个寄存器扮演着不同的角色:

- **ESP (栈指针寄存器)**: 始终指向栈顶, 每次 `push/pop` 操作都会改变 ESP 的值
- **EBP (基址指针寄存器)**: 作为栈帧的基址, 用于稳定地访问参数和局部变量。参数通过 `EBP+偏移` 访问, 局部变量通过 `EBP-偏移` 访问
- **EIP (指令指针寄存器)**: 指向下一条要执行的指令, `call` 指令会修改 EIP 实现跳转, `ret` 指令会恢复 EIP 实现返回
- **EAX (累加寄存器)**: 用于传递函数返回值

3.3 对栈帧结构的掌握

通过实际观察内存窗口的数据变化, 我清晰地看到了栈帧的完整结构: 从高地址到低地址依次是参数、返回地址、旧 EBP、局部变量。参数从右向左入栈的调用约定 (`cdecl`) 也得到了验证。

3.4 对软件安全的意义

理解栈帧结构对软件安全学习至关重要。后续的缓冲区溢出漏洞利用实验, 正是基于

对栈帧布局的深入理解，通过覆盖返回地址来控制程序执行流程。本次实验为后续的安全实验打下了坚实的基础。

3.5 调试技能的提升

通过使用 VC 6.0 的调试功能，我学会了如何设置断点、单步执行、查看反汇编代码、观察寄存器和内存窗口。这些调试技能在后续的程序开发和漏洞分析中都将非常有用。